

FIR syntax

FIR (Functional IR) is the intermediate representation defined in `org.prlprg.fir`. This document gives its textual grammar, which is what `.fir` files, test snapshots, and the parser/printer in `org.prlprg.fir.parseprint` use. The static semantics (typing, ownership, effects, and well-formedness) are described in `FIR.md`.

Notation

Keywords and punctuation are set in bold and typewriter. Nonterminals are set in italics; the single-letter ones are the metavariables below. ε is the empty production. $x_1, \dots, x_n$ is a comma-separated sequence of zero or more x , and $x_1 \dots x_n$ a whitespace-separated one.

- f, x An R name (function name, function parameter name, or named environment variable): an identifier, or any non-empty string in backticks (``...``) with backticks and backslashes escaped.
- r A register name: an identifier (letters, digits, `_`; not `_` alone). Registers are SSA values and are named at their single definition site: a version parameter, a block parameter, or a statement assignee. By convention they don't start with an uppercase letter, so they're visibly distinct from block labels.
- L A basic block label: an identifier.
- i, d An unboxed integer literal (42, -1) and an unboxed real literal (1.5, 1e3). A number is an integer unless it has a decimal point or exponent.
- str A string body, in double quotes with the usual escapes.
- $sexp$ A boxed R value in the SEXP syntax of `org.prlprg.sexp.parseprint`, always delimited by `<` and `>`: e.g. `<int 42>`, `<real 1.5>`, `<sym x>`, `<missing>`, `<nil>`.

Comment lines (`#` to the end of the line) may precede any function, version, block header, or instruction, and are preserved by the printer. Whitespace is insignificant except where it separates tokens.

Modules, functions, versions, and types

$F ::= fn_1 \dots fn_n$	$sig ::= pt_1, \dots, pt_n -fx> t$
$fn ::= prop_1 \dots prop_k \text{ fun } f(x_1, \dots, x_n) \{$	$pt ::= t \mid t @!$
$\quad \quad \quad ver_1 \dots ver_m \}$	$t ::= * \mid q \mid q ?$
$prop ::= @strict \mid @liteSpecial$	$q ::= k \mid p(k \ fx) \mid p?(k \ fx)$
$ver ::= (par_1, \dots, par_n) -fx> t \{ body \}$	$k ::= V \mid B \mid cls \mid dots \mid miss \mid pk$
$\quad \quad \quad \mid (par_1, \dots, par_n) -fx> t \{ \dots \}$	$\quad \quad \quad \mid v(pk)ox \mid v1(pk)ox$
$par ::= \text{reg } r : t \mid \text{reg } r : t @!$	$pk ::= I \mid L \mid R \mid S$
$body ::= blk \ bb_1 \dots bb_n$	$ox ::= \varepsilon \mid s \mid o \mid b \mid f$
$\quad \quad \quad bb ::= L(r_1 : t_1, \dots, r_n : t_n) : blk$	$fx ::= - \mid \sim \mid +$
$\quad \quad \quad blk ::= s_1 ; \dots s_n ; j ;$	

Table 1: Module structure and types

A function has $m \geq 1$ versions; the first is its *baseline* and the rest are specializations, each with the same number of parameters as the function has parameter names. A version with body `...` is a *stub* (a declaration without code, e.g. a builtin). The entry block has no header: a version body starts directly with the entry block's instructions, followed by the labeled blocks. Block parameters ($L(r : t)$) are MLIR-style phis, and jump targets supply their arguments.

Types are read as follows. k is the kind of the value: V is any R value, B an unboxed boolean, cls a closure, $dots$ a `...` list, $miss$ the missing argument, pk an unboxed primitive scalar (integer, logical, real, string), $v(pk)$ a primitive vector, and $v1(pk)$ a primitive vector of length one (a boxed scalar). Only vectors carry an ownership ox : shared (s , the default and never printed), owned, borrowed, or fresh. q adds promiscuity: $p(k \ fx)$ is a promise whose forcing has effects fx and yields a k ; $p?(k \ fx)$ is either such a promise or a k value. A trailing $?$ makes the type *maybe*: a variable of type $q ?$ can hold a value of another type that was written by reflection. $*$ is the top type, sugar for $p?(V +)?$. Effects fx are none ($-$, pure), impure without reflection ($\sim$), or any including reflection ($+$). $@!$ marks a promise-typed parameter as *strict*: the version forces it on every path.

Instructions, expressions, and arguments

$s ::= e \mid r : t = e$	$e ::= \text{noop}$
$j ::= \text{goto } tg$	$\mid \text{mkenv} \mid \text{mkenv}\sim \mid \text{mkenv-} \mid \text{popenv}$
$\mid \text{if } a \text{ then } tg \text{ else } tg$	$\mid \text{ld } x \mid \text{ld-super } x$
$\mid \text{check } tg \text{ else } tg$	$\mid \text{ldf } x \mid \text{ldf-glob } x \mid \text{ldf-base } x$
$\mid \text{deopt } i [a_1, \dots, a_n]$	$\mid \text{st } x = a \mid \text{st-super } x = a$
$\mid \text{return } a \mid \text{raise } a \mid \text{unreachable}$	$\mid a \$x \mid a \$x = a$
$tg ::= L(a_1, \dots, a_n)$	$\mid v(pk)[a_1, \dots, a_n] \mid \text{dots}[na_1, \dots, na_n]$
$a ::= c \mid r \mid \text{consume } r$	$\mid a [a] \mid a [[a]] \mid a [a] = a$
$na ::= a \mid x = a$	$\mid f \text{ dm env } < sig > (a_1, \dots, a_n)$
$on ::= \varepsilon \mid x$	$\mid \text{dyn } a (a_1, \dots, a_n)$
$dm ::= \varepsilon \mid \%$	$\mid \text{dyn } a [on_1, \dots, on_n] (a_1, \dots, a_n)$
$env ::= \varepsilon \mid @a$	$\mid \text{clos } f \mid \text{clos-static } f$
$c ::= i \mid d \mid \text{"str"} \mid <sexp>$	$\mid \text{prom}<t \text{ fx}>\{ \text{body} \}$
$\mid \text{TRUE} \mid \text{FALSE}$	$\mid \text{prom}<t \text{ fx}>\{ \text{body} \}$
$\mid \text{TRUE_LGL} \mid \text{FALSE_LGL} \mid \text{NA_LGL}$	$\mid \text{force } a \mid \text{force? } a$
$\mid \text{NA_INT} \mid \text{NA_REAL} \mid \text{NA_STR}$	$\mid \text{dup } a$
	$\mid a \text{ as } t$
	$\mid a ? : t \mid a ? = c \mid a ? - f$
	$\mid \text{ld } x ? = c \mid \text{ldf } x ? - f$

Table 2: Instructions, expressions, and arguments

A statement is an expression, optionally assigned to a register that is defined and typed right there ($r : t = e$). Every block ends with exactly one jump j . Arguments a are the only sub-expressions: an expression can't nest another expression, only a constant, a register read, or `consume` r , which reads an owned vector for the last time and moves it out. Unboxed constants have unboxed types: i is an I, d an R, a string an S, TRUE/FALSE are B, and TRUE_LGL/FALSE_LGL/NA_LGL are L. Boxed R values are written as $<sexp>$.

The expressions, in the order of Table 2:

- `noop` does nothing (a placeholder the optimizer leaves behind and later removes).
- `mkenv` creates the closure environment; `mkenv~` creates one that's assumed not to be observed by reflection; `mkenv-` is an elided environment that isn't materialized (a `deopt` re-creates it). `popenv` discards the innermost environment.
- `ld` x / `st` $x = a$ load and store a named variable in the innermost environment; the `-super` forms skip it (R's `<<-`). `ldf` x looks x up as a function (forcing promises and skipping non-closures); `ldf-glob` and `ldf-base` start the lookup in the global and base environment.
- $a \$x$ and $a \$x = a$ reflectively load and store x in the environment of the promise a .
- `v(pk)[...]` builds a primitive vector from unboxed elements; `dots[...]` builds a ... list of optionally named elements.
- $a [a]$ reads an element (0-based; out-of-range reads NA); $a [[a]]$ is the same but fails out of range (R's `[]`); $a [a] = a$ writes an element of an owned or fresh vector in place.
- $f \text{ dm env } < sig > (...)$ calls the function f with the given signature. $dm = \%$ makes it a *dispatch* call (the version is chosen at run time); otherwise the version is chosen statically. $env = @a$ passes the closure a whose environment the callee uses; when omitted the closure is elided.
- `dyn` $a (...)$ calls the closure a that is only known at run time; the bracketed list names arguments (an empty `on` leaves one unnamed) and is omitted when none is named.
- `clos` f creates a closure of f over the current environment; `clos-static` f over the global environment.
- `prom` $<t \text{ fx}>\{ \text{body} \}$ creates a promise whose body returns a t with effects fx . The body is a CFG in the enclosing version's register namespace. `prom-` marks the promise *local*: it's speculated not to escape its stack frame, so it may read registers defined outside it.
- `force` a forces a promise; `force?` a forces a if it's a promise and is a no-op otherwise. `dup` a copies a vector. $a \text{ as } t$ casts.
- The $?$ forms are *assumptions*: no-ops when evaluated, checked at the next `check`. $a ? : t$ assumes a has type t , $a ? = c$ that it equals c , $a ? - f$ that it's a closure of f ; `ld` $x ? = c$ and `ldf` $x ? - f$ fuse a load with the corresponding assumption.

The jumps: `goto` is unconditional; `if` branches on a B; `check` checks the assumptions in its first target and jumps to the second (a `deopt` block) if any fails; `deopt` $i [...]$ deoptimizes to GNU-R bytecode position i with the given stack; `return` returns from the version or promise; `raise` throws the S as an R error; `unreachable` marks dead code.